

Chapter 2:

Data Manipulation

Computer Science: An Overview
Eleventh Edition

by
J. Glenn Brookshear

- In Chapter 1 we studied topics relating to the storage of data inside a computer.
- In this chapter we will see how a computer manipulates that data.
- This manipulation consists of moving data from one location to another as well as performing operations such as arithmetic calculations, text editing, and image manipulation.
- We begin by computer architecture beyond that of data storage systems.

Chapter 2: Data Manipulation

- 2.1 Computer Architecture
- 2.2 Machine Language
- 2.3 Program Execution
- 2.4 Arithmetic/Logic Instructions
- 2.5 Communicating with Other Devices
- 2.6 Other Architectures

Computer Architecture

- The circuitry in a computer that controls the manipulation of data is called the **central processing unit, or CPU (often referred to as merely the processor)**.
- The CPUs found in today's desktop computers and notebooks are packaged as small flat squares whose connecting pins plug into a socket mounted on the machine's main circuit board (called the **motherboard**).
- Due to their small size, these processors are called **microprocessors**.

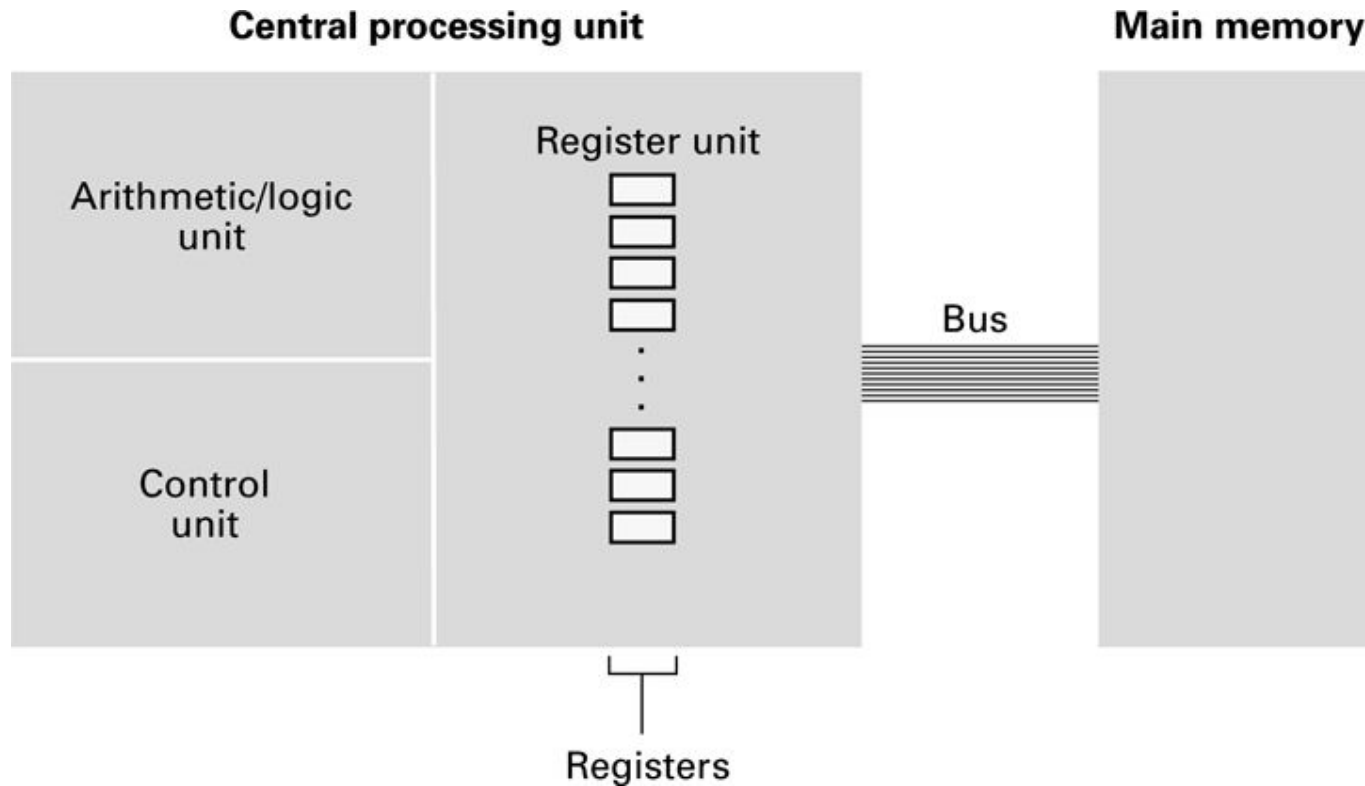
CPU Basics

- A CPU consists of three parts:
- the **arithmetic/logic unit**, which contains the circuitry that performs operations on data (such as addition and subtraction);
- the **control unit**, which contains the circuitry for coordinating the machine's activities;
- and the **register unit**, which contains data storage cells (similar to main memory cells), called **registers**, that are used for temporary storage of information within the CPU.

Computer Architecture

- Central Processing Unit (CPU) or processor
 - **Arithmetic/Logic unit** versus **Control unit**
 - **Registers**
 - General purpose
 - Special purpose
- Bus
- Motherboard

Figure 2.1 CPU and main memory connected via a bus



General-purpose registers

- General-purpose registers serve as temporary holding places for data being manipulated by the CPU.
- These registers hold the inputs to the arithmetic/logic unit's circuitry and provide storage space for results produced by that unit.
- To perform an operation on data stored in main memory, the control unit transfers the data from memory into the general-purpose registers, informs the arithmetic/logic unit which registers hold the data, activates the appropriate circuitry within the arithmetic/logic unit, and tells the arithmetic/logic unit which register should receive the result.

bus

- For the purpose of transferring bit patterns, a machine's CPU and main memory are connected by a collection of wires called a **bus**.
- Through this bus, the CPU extracts (reads) data from main memory by supplying the address of the pertinent memory cell along with an electronic signal telling the memory circuitry that it is supposed to retrieve the data in the indicated cell.
- In a similar manner, the CPU places (writes) data in memory by providing the address of the destination cell and the data to be stored together with the appropriate electronic signal telling main memory that it is supposed to store the data being sent to it.

Figure 2.2 Adding values stored in memory

- Step 1.** Get one of the values to be added from memory and place it in a register.
- Step 2.** Get the other value to be added from memory and place it in another register.
- Step 3.** Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.
- Step 4.** Store the result in memory.
- Step 5.** Stop.

Stored Program Concept

A program can be encoded as bit patterns and stored in main memory. From there, the CPU can then extract the instructions and execute them. In turn, the program to be executed can be altered easily.

Cache Memory

It is instructive to compare the memory facilities within a computer in relation to their functionality. Registers are used to hold the data immediately applicable to the operation at hand; main memory is used to hold data that will be needed in the near future; and mass storage is used to hold data that will likely not be needed in the immediate future. Many machines are designed with an additional memory level, called cache memory. **Cache memory** is a portion (perhaps several hundred KB) of high-speed memory located within the CPU itself. In this special memory area, the machine attempts to keep a copy of that portion of main memory that is of current interest. In this setting, data transfers that normally would be made between registers and main memory are made between registers and cache memory. Any changes made to cache memory are then transferred collectively to main memory at a more opportune time. The result is a CPU that can execute its machine cycle more rapidly because it is not delayed by main memory communication.

1. What sequence of events do you think would be required to move the contents of one memory cell in a computer to another memory cell?
2. What information must the CPU supply to the main memory circuitry to write a value into a memory cell?
3. Mass storage, main memory, and general-purpose registers are all storage systems. What is the difference in their use?

1. What sequence of events do you think would be required to move the contents of one memory cell in a computer to another memory cell?
 2. What information must the CPU supply to the main memory circuitry to write a value into a memory cell?
 3. Mass storage, main memory, and general-purpose registers are all storage systems. What is the difference in their use?
-
1. On some machines this is a two-step process consisting of first reading the contents from the first cell into a register and then writing it from the register into the destination cell. On most machines, this is accomplished as one activity without using an intermediate register.

1. What sequence of events do you think would be required to move the contents of one memory cell in a computer to another memory cell?
2. What information must the CPU supply to the main memory circuitry to write a value into a memory cell?
3. Mass storage, main memory, and general-purpose registers are all storage systems. What is the difference in their use?

2. The value to be written, the address of the cell in which to write, and the command to write

1. What sequence of events do you think would be required to move the contents of one memory cell in a computer to another memory cell?
2. What information must the CPU supply to the main memory circuitry to write a value into a memory cell?
3. Mass storage, main memory, and general-purpose registers are all storage systems. What is the difference in their use?

3. General-purpose registers are used to hold the data immediately applicable to the operation at hand; main memory is used to hold data that will be needed in the near future; and mass storage is used to hold data that will likely not be needed in the near future.

Terminology

- **Machine instruction:** An instruction (or command) encoded as a bit pattern recognizable by the CPU
- **Machine language:** The set of all instructions recognized by a machine

Machine Language Philosophies

CPU architecture

- Reduced Instruction Set Computing (RISC)
 - Few, simple, efficient, and fast instructions
 - Examples: PowerPC from Apple/IBM/Motorola and ARM
- Complex Instruction Set Computing (CISC)
 - Many, convenient, and powerful instructions
 - Example: Intel

Machine Instruction Types

- **Data Transfer:** copy data from one location to another

Step 1. Get one of the values to be added from memory and place it in a register.

Step 2. Get the other value to be added from memory and place it in another register.

Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.

Step 4. Store the result in memory.

Step 5. Stop.

- Steps 1, 2, and 4 fall into this category.
- Special terms are used when referring to the transfer of data between the CPU and main memory.
- A request to fill a general-purpose register with the contents of a memory cell is commonly referred to as a **LOAD** instruction;
- A request to transfer the contents of a register to a memory cell is called a **STORE** instruction.
- Steps 1 and 2 are **LOAD** instructions, and Step 4 is a **STORE** instruction.

Machine Instruction Types

- **Arithmetic/Logic:** The arithmetic/logic group consists of the instructions that tell the control unit to request an activity within the arithmetic/logic unit. Step 3 falls into this group. As its name suggests, the arithmetic/logic unit is capable of performing operations other than the basic arithmetic operations like Boolean operations.
- **Control:** direct the execution of the program

Machine Instruction Types

- Control: The control group consists of those instructions that direct the execution of the program rather than the manipulation of data. Step 5 falls into this category, although it is an extremely elementary example.
- This group contains many of the more interesting instructions in a machine's repertoire, such as the family of JUMP (or BRANCH) instructions used to direct the CPU to execute an instruction other than the next one in the list.

Figure 2.3 Dividing values stored in memory

Step 3 is a conditional jump that protects against the possibility of division by zero

Step 1. LOAD a register with a value from memory.

Step 2. LOAD another register with another value from memory.

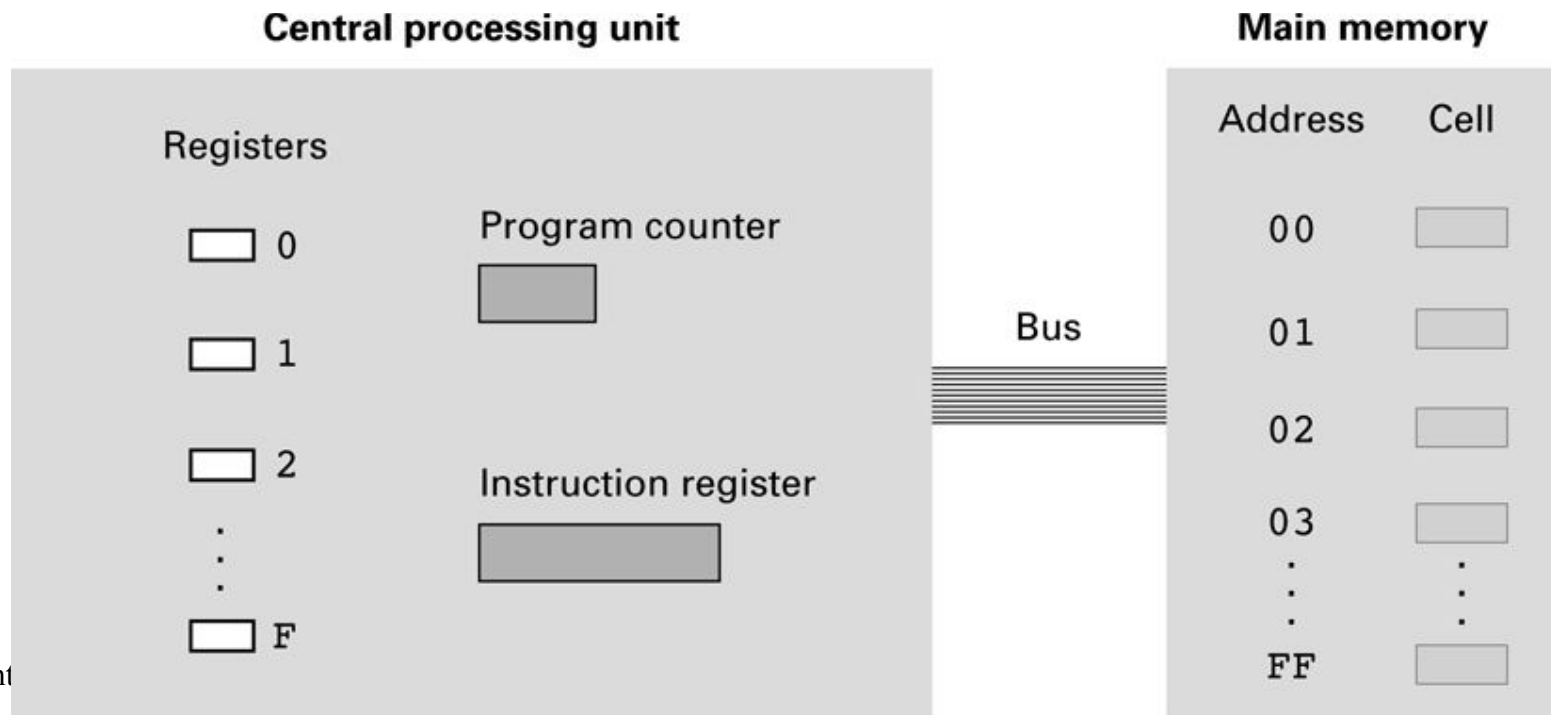
Step 3. If this second value is zero, JUMP to Step 6.

Step 4. Divide the contents of the first register by the second register and leave the result in a third register.

Step 5. STORE the contents of the third register in memory.

Figure 2.4 An Illustrative Machine Language

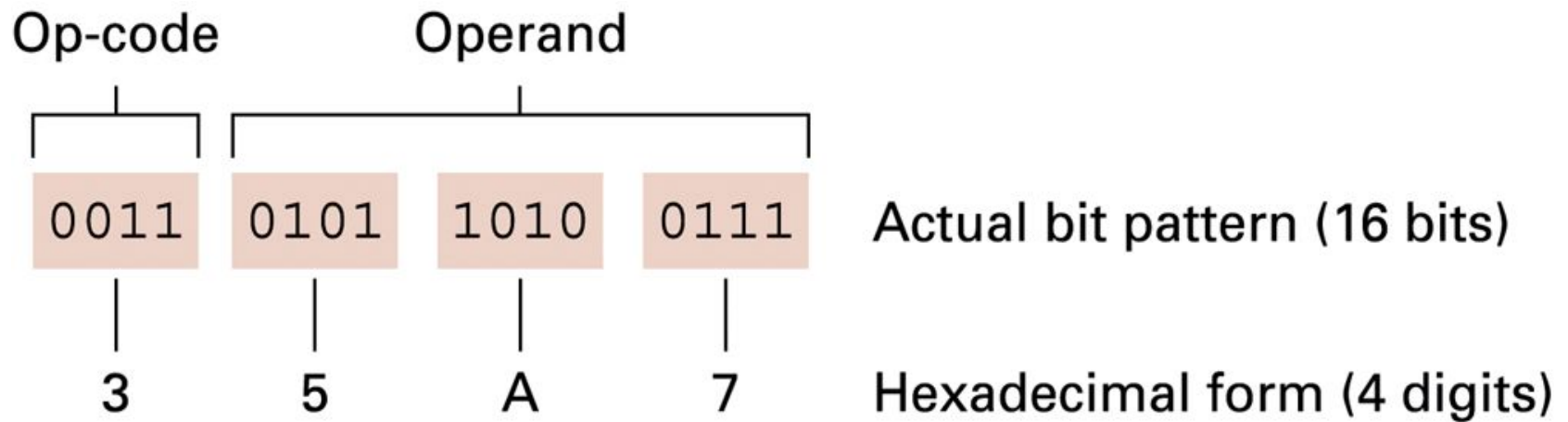
It has 16 general-purpose registers and 256 main memory cells, each with a capacity of 8 bits. For referencing purposes, we label the registers with the values 0 through 15 and address the memory cells with the values 0 through 255.



Parts of a Machine Instruction

- The encoded version of a machine instruction consists of two parts:
- the **op-code**(short for operation code) field and the **operand** field.
- The bit pattern appearing in the op-code field indicates which of the elementary operations, such as STORE, SHIFT, XOR, and JUMP, is requested by the instruction.
- The bit patterns found in the operand field provide more detailed information about the operation specified by the op-code. For example, in the case of a STORE operation, the information in the operand field indicates which register contains the data to be stored and which memory cell is to receive the data.

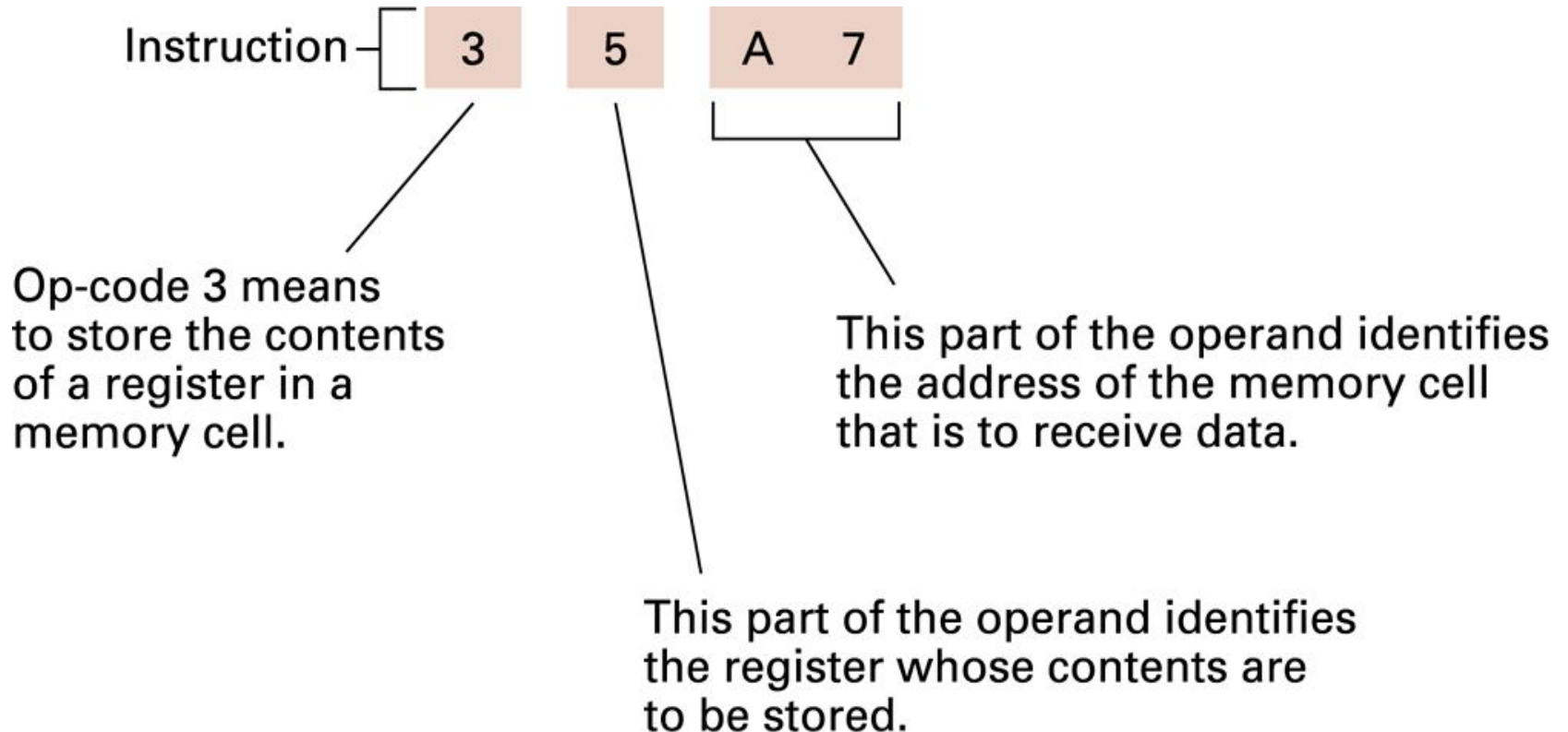
Figure 2.5 The composition of an instruction for the machine in Appendix C



Parts of a Machine Instruction

- The entire machine language of our illustrative machine (Appendix C) consists of only twelve basic instructions.
- Each of these instructions is encoded using a total of 16 bits, represented by four hexadecimal digits
- The op-code for each instruction consists of the first 4 bits or, equivalently, the first hexadecimal digit. Note (Appendix C) that these op-codes are represented by the hexadecimal digits 1 through C.
- The table shows us that an instruction beginning with the hexadecimal digit 3 refers to a STORE instruction, and an instruction beginning with hexadecimal A refers to a ROTATE instruction.

Figure 2.6 Decoding the instruction 35A7



Parts of a Machine Instruction

- The operand field of each instruction in our illustrative machine consists of three hexadecimal digits (12 bits), and clarifies the general instruction given by the op-code.
- For example (Figure 2.6), if the first hexadecimal digit of an instruction were 3 (the op-code for storing the contents of a register), the next hexadecimal digit of the instruction would indicate which register is to be stored, and the last two hexadecimal digits would indicate which memory cell is to receive the data.
- Thus the instruction 35A7 (hexadecimal) translates to the statement “STORE the bit pattern found in register 5 in the memory cell whose address is A7.”

35A7 simplifies is the bit pattern 0011010110100111. 0-28

Figure 2.7 An encoded version of the instructions in Figure 2.2

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

5. The following are instructions written in the machine language described in Appendix C. Rewrite them in English.
- a. 368A b. BADE c. 803C d. 40F4
6. What is the difference between the instructions 15AB and 25AB in the machine language of Appendix C?
7. Here are some instructions in English. Translate each of them into the machine language of Appendix C.
- a. LOAD register number 3 with the hexadecimal value 56.
- b. ROTATE register number 5 three bits to the right.
- c. AND the contents of register A with the contents of register 5 and leave the result in register 0.

5. The following are instructions written in the machine language described in Appendix C. Rewrite them in English.

a. 368A b. BADE c. 803C d. 40F4

6. What is the difference between the instructions 15AB and 25AB in the machine language of Appendix C?

7. Here are some instructions in English. Translate each of them into the machine language of Appendix C.

a. LOAD register number 3 with the hexadecimal value 56.

b. ROTATE register number 5 three bits to the right.

c. AND the contents of register A with the contents of register 5 and leave the result in register 0.

5. a. STORE the contents of register 6 in memory cell number 8A.

b. JUMP to location DE if the contents of register A equals that of register 0.

c. AND the contents of registers 3 and C, leaving the result in register 0.

d. MOVE the contents of register F to register 4.

5. The following are instructions written in the machine language described in Appendix C. Rewrite them in English.
- a. 368A b. BADE c. 803C d. 40F4
6. What is the difference between the instructions 15AB and 25AB in the machine language of Appendix C?
7. Here are some instructions in English. Translate each of them into the machine language of Appendix C.
- a. LOAD register number 3 with the hexadecimal value 56.
b. ROTATE register number 5 three bits to the right.
c. AND the contents of register A with the contents of register 5 and leave the result in register 0.

6. The instruction 15AB requires that the CPU query the memory circuitry for the contents of the memory cell at address AB. This value, when obtained from memory, is then placed in register 5. The instruction 25AB does not require such a request of memory. Rather, the value AB is placed in register 5.

5. The following are instructions written in the machine language described in Appendix C. Rewrite them in English.
- a. 368A b. BADE c. 803C d. 40F4
6. What is the difference between the instructions 15AB and 25AB in the machine language of Appendix C?
7. Here are some instructions in English. Translate each of them into the machine language of Appendix C.
- a. LOAD register number 3 with the hexadecimal value 56.
b. ROTATE register number 5 three bits to the right.
c. AND the contents of register A with the contents of register 5 and leave the result in register 0.

7. a. 2356 b. A503 c. 80A5

Program Execution

- A computer follows a program stored in its memory by copying the instructions from memory into the CPU as needed.
- Once in the CPU, each instruction is decoded.
- The order in which the instructions are fetched from memory corresponds to the order in which the instructions are stored in memory unless otherwise altered by a JUMP instruction.

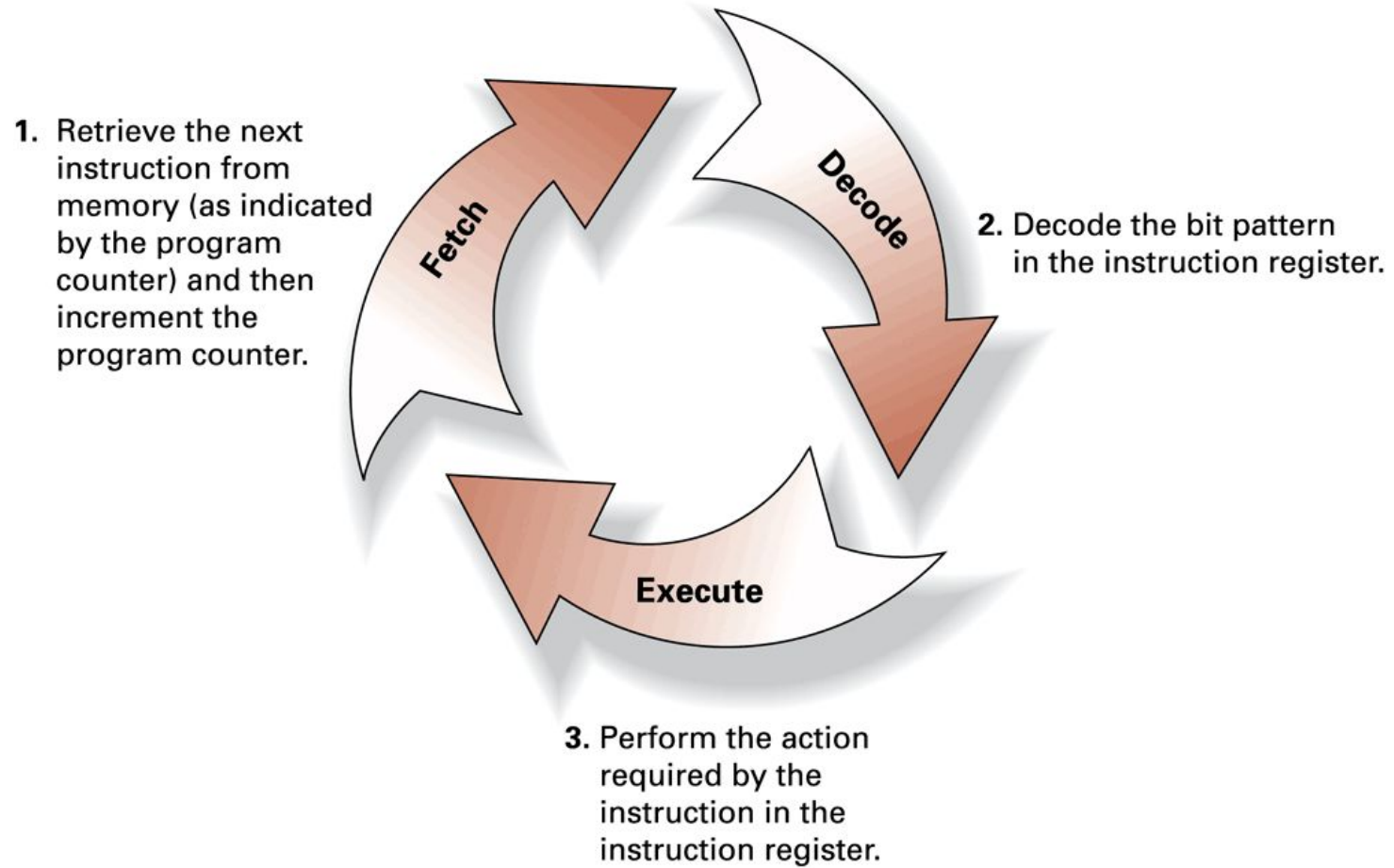
Program Execution

- To understand how the overall execution process takes place, it is necessary to consider two of the special purpose registers within the CPU: the **instruction register** and the **program counter** (see again Figure 2.4).
- The **instruction register** is used to hold the instruction being executed.
- The **program counter** contains the address of the next instruction to be executed, thereby serving as the machine's way of keeping track of where it is in the program.

Program Execution

- The CPU performs its job by continually repeating an algorithm that guides it through a three-step process known as the **machine cycle**.
- The steps in the machine cycle are fetch, decode, and execute (Figure 2.8).
- During the fetch step, the CPU requests that main memory provide it with the instruction that is stored at the address indicated by the program counter.
- Since each instruction in our machine is two bytes long, this fetch process involves retrieving the contents of two memory cells from main memory.

Figure 2.8 The machine cycle



Program Execution

- The CPU places the instruction received from memory in its instruction register and then increments the program counter by two so that the counter contains the address of the next instruction stored in memory. Thus the program counter will be ready for the next fetch.
- With the instruction now in the instruction register, the CPU decodes the instruction, which involves breaking the operand field into its proper components based on the instruction's op-code.

Program Execution

- The CPU then executes the instruction by activating the appropriate circuitry to perform the requested task.
- For example, if the instruction is a load from memory, the CPU sends the appropriate signals to main memory, waits for main memory to send the data, and then places the data in the requested register;
- if the instruction is for an arithmetic operation, the CPU activates the appropriate circuitry in the arithmetic/logic unit with the correct registers as inputs and waits for the arithmetic/logic unit to compute the answer and place it in the appropriate register.

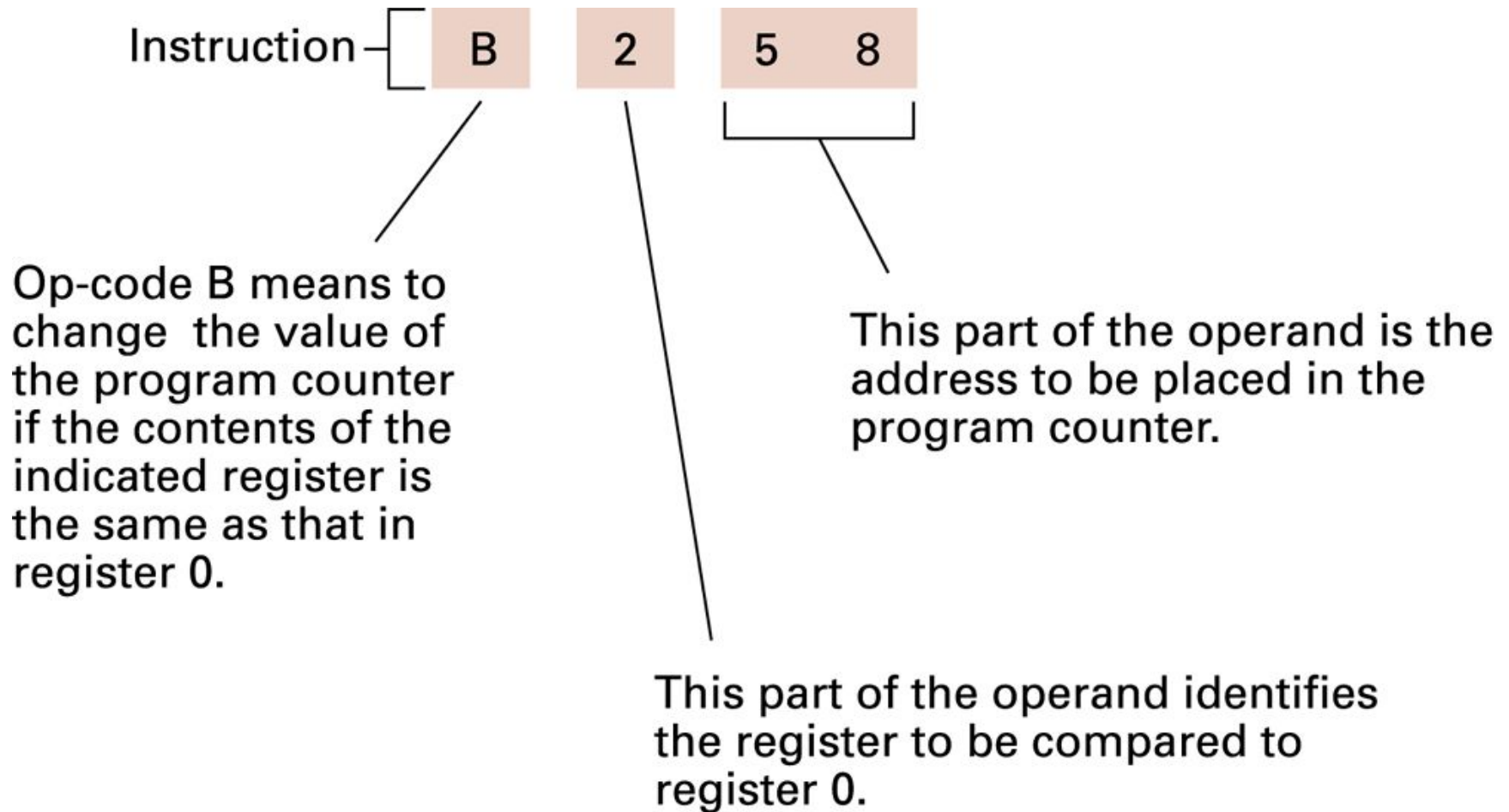
Program Execution

- Once the instruction in the instruction register has been executed, the CPU again begins the machine cycle with the fetch step.
- Observe that since the program counter was incremented at the end of the previous fetch, it again provides the CPU with the correct address.

Program Execution

- A somewhat special case is the execution of a JUMP instruction.
- Consider, for example, the instruction B258 (Figure 2.9), which means “JUMP to the instruction at address 58 (hexadecimal) if the contents of register 2 is the same as that of register 0.”
- In this case, the execute step of the machine cycle begins with the comparison of registers 2 and 0.
- If they contain different bit patterns, the execute step terminates and the next machine cycle begins.

Figure 2.9 Decoding the instruction B258



Program Execution

- If, however, the contents of these registers are equal, the machine places the value 58 (hexadecimal) in its program counter during the execute step.
- In this case, then, the next fetch step finds 58 in the program counter, so the instruction at that address will be the next instruction to be fetched and executed.

Comparing Computer Power

When shopping for a personal computer, you will find that clock speeds are often used to compare machines. A computer's **clock** is a circuit, called an oscillator, which generates pulses that are used to coordinate the machine's activities—the faster this oscillating circuit generates pulses, the faster the machine performs its machine cycle. Clock speeds are measured in hertz (abbreviated as Hz) with one Hz equal to one cycle (or pulse) per second. Typical clock speeds in desktop computers are in the range of a few hundred MHz (older models) to several GHz. (MHz is short for megahertz, which is a million Hz. GHz is short for gigahertz, which is 1000 MHz.)

Unfortunately, different CPU designs might perform different amounts of work in one clock cycle, and thus clock speed alone fails to be relevant in comparing machines with different CPUs. If you are comparing a machine based on an Intel processor to one based on ARM, it would be more meaningful to compare performance by means of **benchmarking**, which is the process of comparing the performance of different machines when executing the same program, known as a benchmark. By selecting benchmarks representing different types of applications, you get meaningful comparisons for various market segments.

An Example of Program Execution

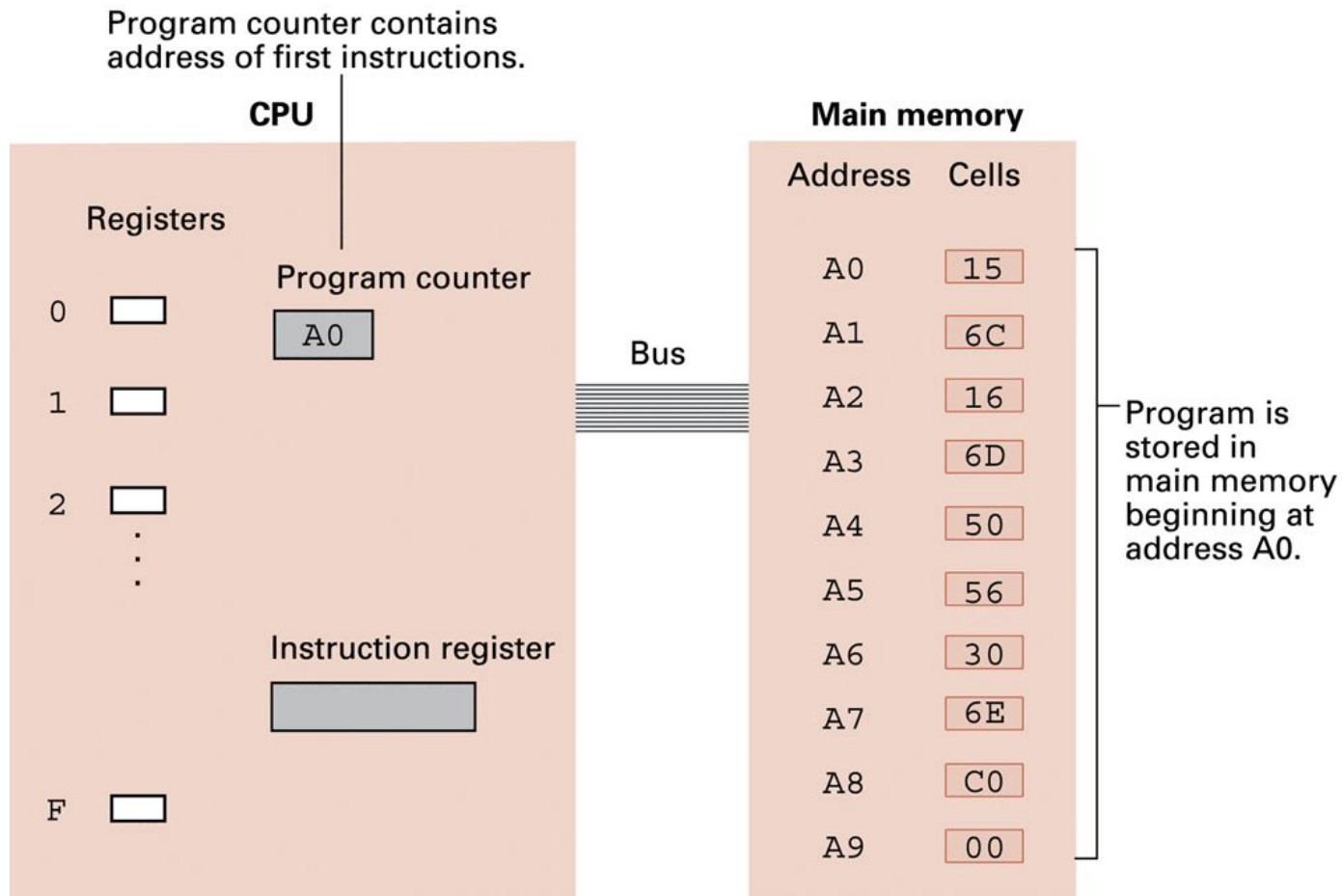
- Let us follow the machine cycle applied to the program presented in Figure 2.7, which retrieves two values from main memory, computes their sum, and stores that total in a main memory cell.

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

An Example of Program Execution

- We first need to put the program somewhere in memory. For our example, suppose the program is stored in consecutive addresses, starting at address A0 (hexadecimal).
- With the program stored in this manner, we can cause the machine to execute it by placing the address (A0) of the first instruction in the program counter and starting the machine (Figure 2.10).

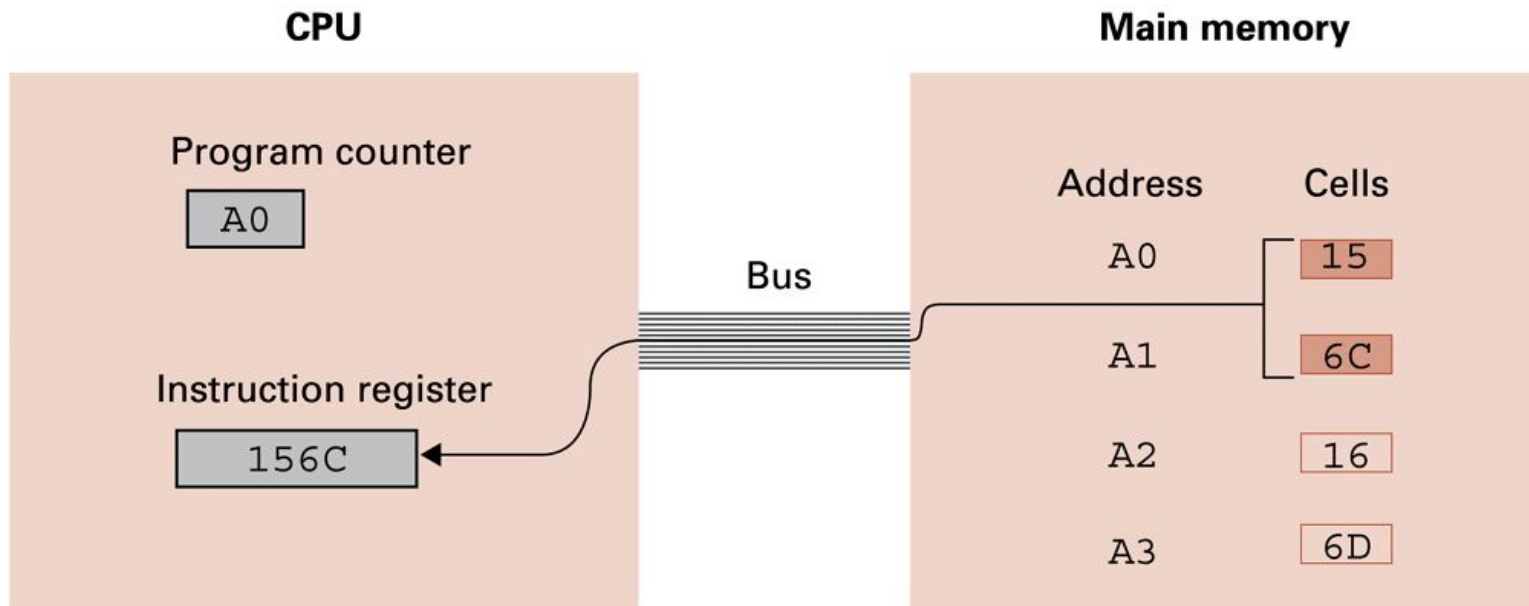
Figure 2.10 The program from Figure 2.7 stored in main memory ready for execution



An Example of Program Execution

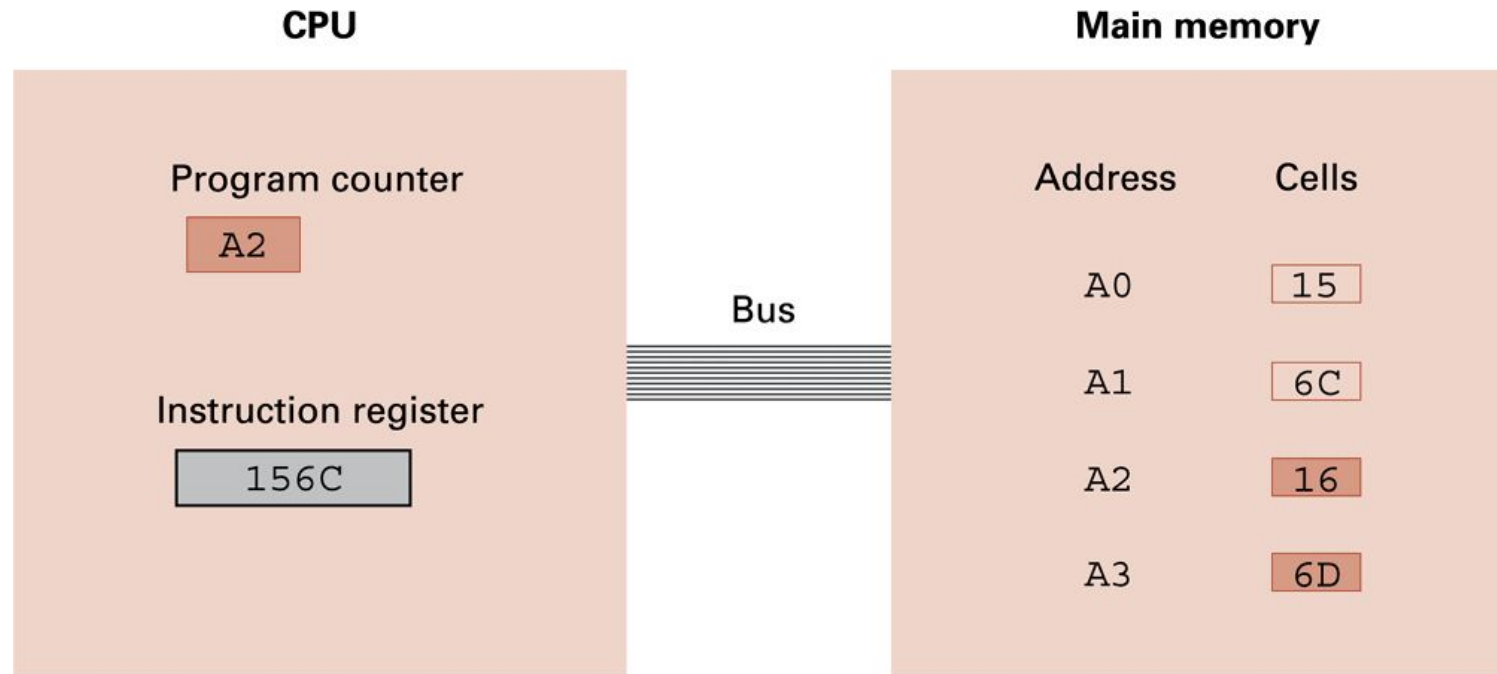
- The CPU begins the fetch step of the machine cycle by extracting the instruction stored in main memory at location A0 and placing this instruction (156C) in its instruction register (Figure 2.11a).
- Notice that, in our machine, instructions are 16 bits (two bytes) long. Thus the entire instruction to be fetched occupies the memory cells at both address A0 and A1. So it retrieves the contents of both cells and places the bit patterns received in the instruction register, which is 16 bits long.
- The CPU then adds two to the program counter so that this register contains the address of the next instruction (Figure 2.11b).

Figure 2.11 Performing the fetch step of the machine cycle



- a. At the beginning of the fetch step the instruction starting at address A0 is retrieved from memory and placed in the instruction register.

Figure 2.11 Performing the fetch step of the machine cycle (cont'd)



b. Then the program counter is incremented so that it points to the next instruction.

An Example of Program Execution

- At the end of the fetch step of the first machine cycle, the program counter and instruction register contain the following data:
 - Program Counter: A2
 - Instruction Register: 156C
- Next, the CPU analyzes the instruction in its instruction register and concludes that it is to load register 5 with the contents of the memory cell at address 6C.
- This load activity is performed during the execution step of the machine cycle, and the CPU then begins the next cycle.

An Example of Program Execution

- This cycle begins by fetching the instruction 166D from the two memory cells starting at address A2.
- The CPU places this instruction in the instruction register and increments the program counter to A4.
- The values in the program counter and instruction register therefore become the following:
 - Program Counter: A4
 - Instruction Register: 166D
- Now the CPU decodes the instruction 166D and determines that it is to load register 6 with the contents of memory address 6D.
- It then executes the instruction. It is at this time that register 6 is actually loaded.

An Example of Program Execution

- Since the program counter now contains A4, the CPU extracts the next instruction starting at this address.
- The result is that 5056 is placed in the instruction register, and the program counter is incremented to A6.
- The CPU now decodes the contents of its instruction register and executes it by activating the two's complement addition circuitry with inputs being registers 5 and 6.

An Example of Program Execution

- During this execution step, the arithmetic/logic unit performs the requested addition, leaves the result in register 0 (as requested by the control unit), and reports to the control unit that it has finished.
- The CPU then begins another machine cycle. Once again, with the aid of the program counter, it fetches the next instruction (306E) from the two memory cells starting at memory location A6 and increments the program counter to A8.
- This instruction is then decoded and executed.
- At this point, the sum is placed in memory location 6E.

An Example of Program Execution

- The next instruction is fetched starting from memory location A8, and the program counter is incremented to AA.
- The contents of the instruction register (C000) are now decoded as the halt instruction.
- Consequently, the machine stops during the execute step of the machine cycle, and the program is completed.

Programs Versus Data

- Because data are also contained in main memory and encoded in terms of 0s and 1s, the machine alone has no way of knowing what is data and what is program.
- If the program counter were assigned the address of data instead of the address of the desired program, the CPU, not knowing any better, would extract the data bit patterns as though they were instructions and execute them.
- The final result would depend on the data involved.

1. Suppose the memory cells from addresses 00 to 05 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
00	14
01	02
02	34
03	17
04	C0
05	00

If we start the machine with its program counter containing 00, what bit pattern is in the memory cell whose address is hexadecimal 17 when the machine halts?

1. Suppose the memory cells from addresses 00 to 05 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
00	14
01	02
02	34
03	17
04	C0
05	00

Hexadecimal 34

If we start the machine with its program counter containing 00, what bit pattern is in the memory cell whose address is hexadecimal 17 when the machine halts?

Op-code	Operand	Description
1	RXY	LOAD the register R with the bit pattern found in the memory cell whose address is XY. <i>Example:</i> 14A3 would cause the contents of the memory cell located at address A3 to be placed in register 4.
3	RXY	STORE the bit pattern found in register R in the memory cell whose address is XY. <i>Example:</i> 35B1 would cause the contents of register 5 to be placed in the memory cell whose address is B1.

2. Suppose the memory cells at addresses B0 to B8 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
B0	13
B1	B8
B2	A3
B3	02
B4	33
B5	B8
B6	C0
B7	00
B8	0F

- If the program counter starts at B0, what bit pattern is in register number 3 after the first instruction has been executed?
- What bit pattern is in memory cell B8 when the halt instruction is executed?

2. Suppose the memory cells at addresses B0 to B8 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
B0	13
B1	B8
B2	A3
B3	02
B4	33
B5	B8
B6	C0
B7	00
B8	0F

- a. If the program counter starts at B0, what bit pattern is in register number 3 after the first instruction has been executed?
- b. What bit pattern is in memory cell B8 when the halt instruction is executed?

a. 0F b. C3

00001111->11000011

3. Suppose the memory cells at addresses A4 to B1 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
A4	20
A5	00
A6	21
A7	03
A8	22
A9	01
AA	B1
AB	B0
AC	50
AD	02
AE	B0
AF	AA
B0	C0
B1	00

When answering the following questions, assume that the machine is started with its program counter containing A4.

- What is in register 0 the first time the instruction at address AA is executed?
- What is in register 0 the second time the instruction at address AA is executed?
- How many times is the instruction at address AA executed before the machine halts?

3. Suppose the memory cells at addresses A4 to B1 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
A4	20
A5	00
A6	21
A7	03
A8	22
A9	01
AA	B1
AB	B0
AC	50
AD	02
AE	B0
AF	AA
B0	C0
B1	00

a. 00 b. 01 c. four times

When answering the following questions, assume that the machine is started with its program counter containing A4.

- a. What is in register 0 the first time the instruction at address AA is executed?
- b. What is in register 0 the second time the instruction at address AA is executed?
- c. How many times is the instruction at address AA executed before the machine halts?

2.4 Arithmetic/Logic Instructions

As indicated earlier, the arithmetic/logic group of instructions consists of instructions requesting arithmetic, logic, and shift operations. In this section, we look at these operations more closely.

Logic Operations

We introduced the logic operations AND, OR, and XOR (exclusive or) in Chapter 1 as operations that combine two input bits to produce a single output bit. These operations can be extended to operations that combine two strings of bits to produce a single output string by applying the basic operation to individual columns. For example, the result of ANDing the patterns 10011010 and 11001001 results in

$$\begin{array}{r} 10011010 \\ \text{AND } 11001001 \\ \hline 10001000 \end{array}$$

where we have merely written the result of ANDing the 2 bits in each column at the bottom of the column. Likewise, ORing and XORing these patterns would produce

$$\begin{array}{r} 10011010 \\ \text{OR } 11001001 \\ \hline 11011011 \end{array} \qquad \begin{array}{r} 10011010 \\ \text{XOR } 11001001 \\ \hline 01010011 \end{array}$$

One of the major uses of the AND operation is for placing 0s in one part of a bit pattern while not disturbing the other part. Consider, for example, what happens if the byte 00001111 is the first operand of an AND operation. Without knowing the contents of the second operand, we still can conclude that the four most significant bits of the result will be 0s. Moreover, the four least significant bits of

the result will be a copy of that part of the second operand, as shown in the following example:

```
      00001111
AND 10101010
-----
      00001010
```

This use of the AND operation is an example of the process called **masking**. Here one operand, called a **mask**, determines which part of the other operand will affect the result. In the case of the AND operation, masking produces a result that is a partial replica of one of the operands, with 0s occupying the nonduplicated positions.

Where the AND operation can be used to duplicate a part of a bit string while placing 0s in the nonduplicated part, the OR operation can be used to duplicate a part of a string while putting 1s in the nonduplicated part. For this we again use a mask, but this time we indicate the bit positions to be duplicated with 0s and use 1s to indicate the nonduplicated positions. For example, ORing any byte with 11110000 produces a result with 1s in its most significant 4 bits while its remaining bits are a copy of the least significant 4 bits of the other operand, as demonstrated by the following example:

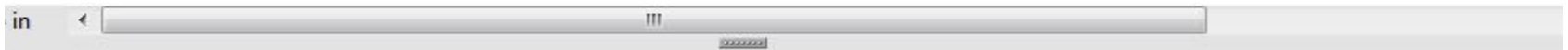
```
      11110000
OR  10101010
-----
      11111010
```

Consequently, whereas the mask 11011111 can be used with the AND operation to force a 0 in the third bit from the high-order end of a byte, the mask 00100000 can be used with the OR operation to force a 1 in that position.

Arithmetic/Logic Operations

A major use of the XOR operation is in forming the complement of a bit string. XORing any byte with a mask of all 1s produces the complement of the byte. For example, note the relationship between the second operand and the result in the following example:

```
    11111111
XOR 10101010
-----
    01010101
```

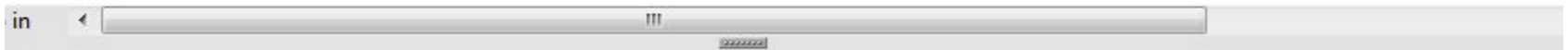


In the machine language described in Appendix C, op-codes 7, 8, and 9 are used for the logic operations OR, AND, and XOR, respectively. Each requests that the corresponding logic operation be performed between the contents of two designated registers and that the result be placed in another designated register. For example, the instruction 7ABC requests that the result of ORing the contents of registers B and C be placed in register A.

Arithmetic/Logic Operations

A major use of the XOR operation is in forming the complement of a bit string. XORing any byte with a mask of all 1s produces the complement of the byte. For example, note the relationship between the second operand and the result in the following example:

```
    11111111
XOR 10101010
-----
    01010101
```



In the machine language described in Appendix C, op-codes 7, 8, and 9 are used for the logic operations OR, AND, and XOR, respectively. Each requests that the corresponding logic operation be performed between the contents of two designated registers and that the result be placed in another designated register. For example, the instruction 7ABC requests that the result of ORing the contents of registers B and C be placed in register A.

Rotation and Shift Operations

- The operations in the class of rotation and shift operations provide a means for moving bits within a register and are often used in solving alignment problems.
- These operations are classified by the direction of motion (right or left) and whether the process is circular.
- Consider a register containing a byte of bits. If we shift its contents 1 bit to the right, we imagine the rightmost bit falling off the edge and a hole appearing at the leftmost end.

Rotation and Shift Operations

- What happens with this extra bit and the hole is the distinguishing feature among the various shift operations.
- One technique is to place the bit that fell off the right end in the hole at the left end.
- The result is a **circular shift**, also called a **rotation**.
- Thus, if we perform a right circular shift on a byte-size bit pattern eight times, we obtain the same bit pattern we started with.

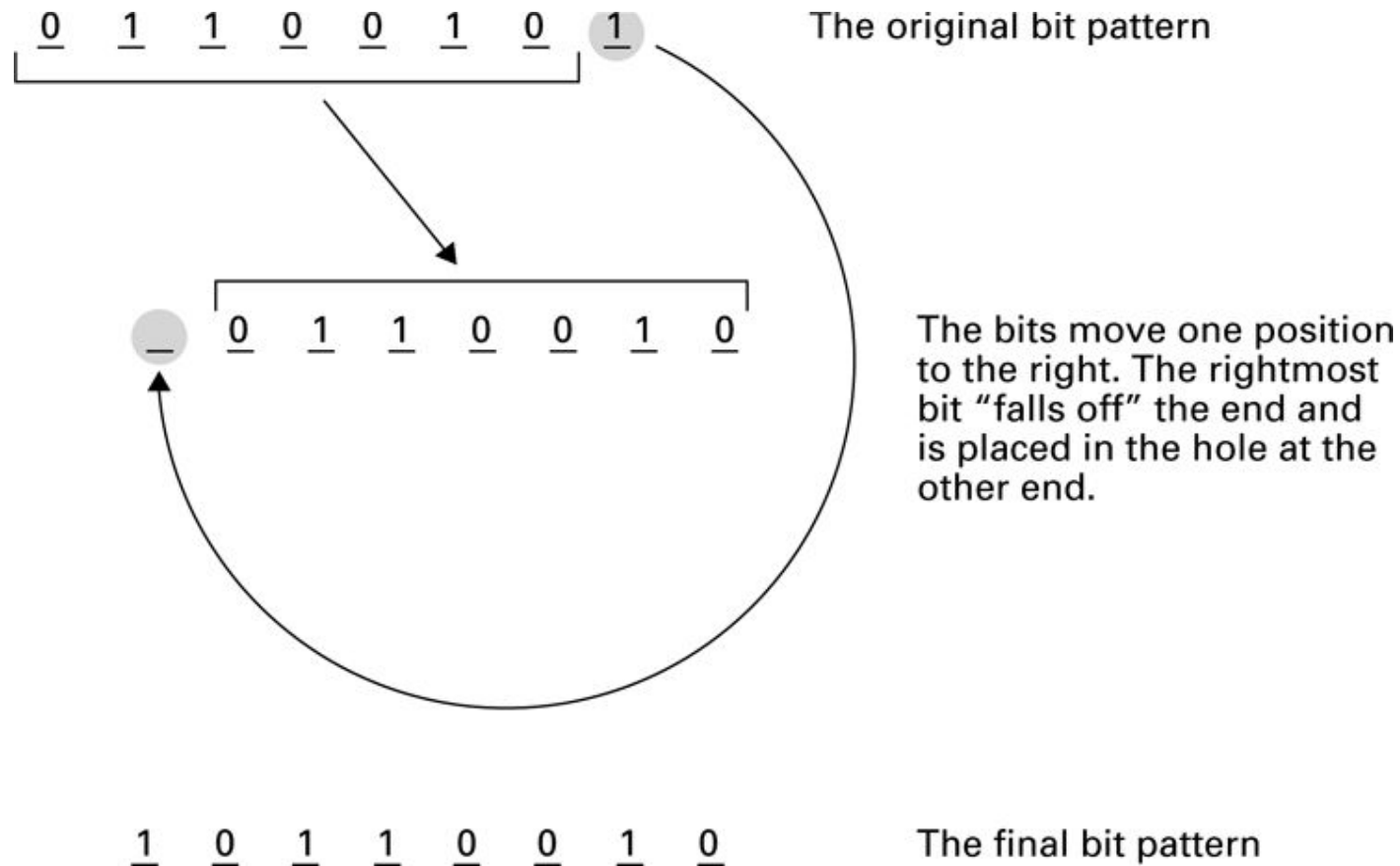
Rotation and Shift Operations

- Another technique is to discard the bit that falls off the edge and always fill the hole with a 0. The term **logical shift** is often used to refer to these operations.
- Such shifts to the left can be used for multiplying.
- Moreover, division by two can be accomplished by shifting the binary string to the right.
- In either shift, care must be taken to preserve the sign bit when using certain notational systems.
- Shifts that leave the sign bit unchanged are sometimes called **arithmetic shifts**.

Rotation and Shift Operations

- Among the variety of shift and rotate instructions possible, the machine language described in Appendix C contains only a right circular shift, designated by op-code A.
- In this case the first hexadecimal digit in the operand specifies the register to be rotated, and the rest of the operand specifies the number of bits to be rotated.
- Thus the instruction A501 means “Rotate the contents of register 5 to the right by 1 bit.”
- In particular, if register 5 originally contained the bit pattern 65 (hexadecimal), then it would contain B2 after this instruction is executed (Figure 2.12).

Figure 2.12 Rotating the bit pattern 65 (hexadecimal) one bit to the right



1. Perform the indicated operations.

a. 01001011
 AND 10101011

b. 10000011
 AND 11101100

c. 11111111
 AND 00101101

d. 01001011
 OR 10101011

e. 10000011
 OR 11101100

f. 11111111
 OR 00101101

g. 01001011
 XOR 10101011

h. 10000011
 XOR 11101100

i. 11111111
 XOR 00101101

2. Suppose you want to isolate the middle 4 bits of a byte by placing 0s in the other 4 bits without disturbing the middle 4 bits. What mask must you use together with what operation?

1. Perform the indicated operations.

a.
$$\begin{array}{r} 01001011 \\ \text{AND } 10101011 \\ \hline \end{array}$$

b.
$$\begin{array}{r} 10000011 \\ \text{AND } 11101100 \\ \hline \end{array}$$

c.
$$\begin{array}{r} 11111111 \\ \text{AND } 00101101 \\ \hline \end{array}$$

d.
$$\begin{array}{r} 01001011 \\ \text{OR } 10101011 \\ \hline \end{array}$$

e.
$$\begin{array}{r} 10000011 \\ \text{OR } 11101100 \\ \hline \end{array}$$

f.
$$\begin{array}{r} 11111111 \\ \text{OR } 00101101 \\ \hline \end{array}$$

g.
$$\begin{array}{r} 01001011 \\ \text{XOR } 10101011 \\ \hline \end{array}$$

h.
$$\begin{array}{r} 10000011 \\ \text{XOR } 11101100 \\ \hline \end{array}$$

i.
$$\begin{array}{r} 11111111 \\ \text{XOR } 00101101 \\ \hline \end{array}$$

2. Suppose you want to isolate the middle 4 bits of a byte by placing 0s in the other 4 bits without disturbing the middle 4 bits. What mask must you use together with what operation?

1. a. 00001011

b. 10000000

c. 00101101

d. 11101011

e. 11101111

f. 11111111

g. 11100000

h. 01101111

i. 11010010

1. Perform the indicated operations.

a.
$$\begin{array}{r} 01001011 \\ \text{AND } 10101011 \\ \hline \end{array}$$

b.
$$\begin{array}{r} 10000011 \\ \text{AND } 11101100 \\ \hline \end{array}$$

c.
$$\begin{array}{r} 11111111 \\ \text{AND } 00101101 \\ \hline \end{array}$$

d.
$$\begin{array}{r} 01001011 \\ \text{OR } 10101011 \\ \hline \end{array}$$

e.
$$\begin{array}{r} 10000011 \\ \text{OR } 11101100 \\ \hline \end{array}$$

f.
$$\begin{array}{r} 11111111 \\ \text{OR } 00101101 \\ \hline \end{array}$$

g.
$$\begin{array}{r} 01001011 \\ \text{XOR } 10101011 \\ \hline \end{array}$$

h.
$$\begin{array}{r} 10000011 \\ \text{XOR } 11101100 \\ \hline \end{array}$$

i.
$$\begin{array}{r} 11111111 \\ \text{XOR } 00101101 \\ \hline \end{array}$$

2. Suppose you want to isolate the middle 4 bits of a byte by placing 0s in the other 4 bits without disturbing the middle 4 bits. What mask must you use together with what operation?

2. 00111100 with the AND operation

3. Suppose you want to complement the 4 middle bits of a byte while leaving the other 4 bits undisturbed. What mask must you use together with what operation?

3. Suppose you want to complement the 4 middle bits of a byte while leaving the other 4 bits undisturbed. What mask must you use together with what operation?

3. 00111100 with the XOR operation

5. It is often convenient to use a logical operation in place of a numeric one. For example, the logical operation AND combines 2 bits in the same manner as multiplication. Which logical operation is almost the same as adding 2 bits, and what goes wrong in this case?

5. It is often convenient to use a logical operation in place of a numeric one. For example, the logical operation AND combines 2 bits in the same manner as multiplication. Which logical operation is almost the same as adding 2 bits, and what goes wrong in this case?

5. The logical XOR operation mirrors addition except for the case where both operands are 1, in which case the XOR produces a 0, whereas the sum is 10. (Thus the XOR operation can be considered an addition operation with no carry.)

6. What logical operation together with what mask can you use to change ASCII codes of lowercase letters to uppercase? What about uppercase to lowercase?
7. What is the result of performing a 3-bit right circular shift on the following bit strings:
 - a. 01101010 b. 00001111 c. 01111111

6. What logical operation together with what mask can you use to change ASCII codes of lowercase letters to uppercase? What about uppercase to lowercase?
7. What is the result of performing a 3-bit right circular shift on the following bit strings:
 - a. 01101010 b. 00001111 c. 01111111

6. Use AND with the mask 11011111 to change lowercase to uppercase. Use OR with 00100000 to change uppercase to lowercase.

6. What logical operation together with what mask can you use to change ASCII codes of lowercase letters to uppercase? What about uppercase to lowercase?
7. What is the result of performing a 3-bit right circular shift on the following bit strings:
 - a. 01101010 b. 00001111 c. 01111111

7. a. 01001101 b. 11100001 c. 11101111

8. What is the result of performing a 1-bit left circular shift on the following bytes represented in hexadecimal notation? Give your answer in hexadecimal form.
- a. AB b. 5C c. B7 d. 35
9. A right circular shift of 3 bits on a string of 8 bits is equivalent to a left circular shift of how many bits?

8. What is the result of performing a 1-bit left circular shift on the following bytes represented in hexadecimal notation? Give your answer in hexadecimal form.

a. AB

b. 5C

c. B7

d. 35

9. A right circular shift of 3 bits on a string of 8 bits is equivalent to a left circular shift of how many bits?

8. a. 57

b. B8

c. 6F

d. 6A

8. What is the result of performing a 1-bit left circular shift on the following bytes represented in hexadecimal notation? Give your answer in hexadecimal form.

a. AB

b. 5C

c. B7

d. 35

9. A right circular shift of 3 bits on a string of 8 bits is equivalent to a left circular shift of how many bits?

9. 5

11. Using the machine language of Appendix C, write a program that places a 1 in the most significant bit of the memory cell whose address is A7 without modifying the remaining bits in the cell.
12. Using the machine language of Appendix C, write a program that copies the middle 4 bits from memory cell E0 into the least significant 4 bits of memory cell E1, while placing 0s in the most significant 4 bits of the cell at location E1.

11. Using the machine language of Appendix C, write a program that places a 1 in the most significant bit of the memory cell whose address is A7 without modifying the remaining bits in the cell.
12. Using the machine language of Appendix C, write a program that copies the middle 4 bits from memory cell E0 into the least significant 4 bits of memory cell E1, while placing 0s in the most significant 4 bits of the cell at location E1.

1. One solution is as follows:

12A7 (LOAD register 2 with the contents of memory cell A7.)

2380 (LOAD register 3 with the value 80.)

7023 (OR registers 2 and 3 leaving the result in register 0.)

30A7 (STORE contents of register 0 in memory cell A7.)

C000 (HALT.)

11. Using the machine language of Appendix C, write a program that places a 1 in the most significant bit of the memory cell whose address is A7 without modifying the remaining bits in the cell.
12. Using the machine language of Appendix C, write a program that copies the middle 4 bits from memory cell E0 into the least significant 4 bits of memory cell E1, while placing 0s in the most significant 4 bits of the cell at location E1.

2. One solution is as follows:

15E0 (LOAD register 5 with the contents of memory cell E0.)
A502 (ROTATE the contents of register 5 to the left by 2 bits.)
260F (LOAD register 6 with the value 0F.)
8056 (AND registers 5 and 6, leaving the result in register 0.)
30E1 (STORE the contents of register 0 in memory cell E1.)
C000 (HALT.)

Communicating with Other Devices

- Main memory and the CPU form the core of a computer. In this section, we investigate how this core, which we will refer to as the computer, communicates with peripheral devices such as mass storage systems, printers, keyboards, mice, display screens, digital cameras, and even other computers.

Communicating with Other Devices

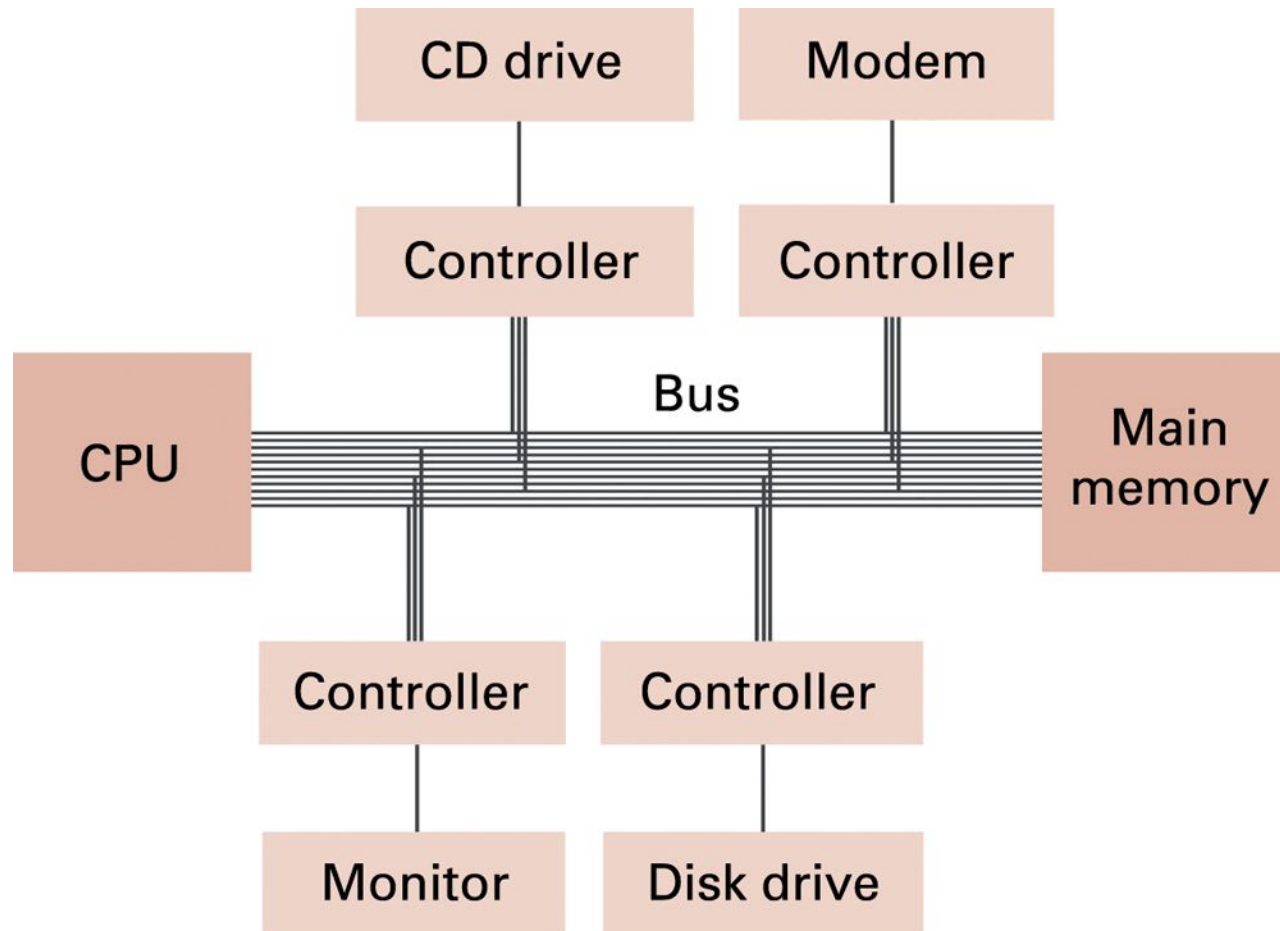
The Role of **Controllers**

- Communication between a computer and other devices is normally handled through an intermediary apparatus known as a **controller**.
- In the case of a personal computer, a controller may consist of circuitry permanently mounted on the computer's motherboard
- The controller connects via cables to peripheral devices within the computer case or perhaps to
- a connector, called a **port**, on the back of the computer where external devices can be attached.

Communicating with Other Devices

- Each controller communicates with the computer itself by means of connections to the same bus that connects the computer's CPU and main memory (Figure 2.13).
- From this position it is able to monitor the signals being sent between the CPU and main memory as well as to inject its own signals onto the bus.
- With this arrangement, the CPU is able to communicate with the controllers attached to the bus in the same manner that it communicates with main memory.

Figure 2.13 Controllers attached to a machine's bus



Communicating with Other Devices

- To send a bit pattern to a controller, the bit pattern is first constructed in one of the CPU's general-purpose registers.
- Then an instruction similar to a STORE instruction is executed by the CPU to “store” the bit pattern in the controller.
- Likewise, to receive a bit pattern from a controller, an instruction similar to a LOAD instruction is used.

Communicating with Other Devices

Direct Memory Access

- Since a controller is attached to a computer's bus, it can carry on its own communication with main memory during those nanoseconds in which the CPU is not using the bus. This ability of a controller to access main memory is known as **direct memory access (DMA)**, and it is a significant asset to a computer's performance.
- The CPU can then continue with other tasks while the controller performs the read operation and deposits the data in main memory via DMA. Thus two activities will be performed at the same time.

Communicating with Other Devices (continued)

- **Handshaking:** The transfer of data between two computer components is rarely a one-way affair.
- Even though we may think of a printer as a device that receives data, the truth is that a printer also sends data back to the computer.
- After all, a computer can produce and send characters to a printer much faster than the printer can print them. If a computer blindly sent data to a printer, the printer would quickly fall behind, resulting in lost data. Thus a process such as printing a document involves a constant two-way dialogue, known as **handshaking**, in which the computer and the peripheral device exchange information about the device's status.

Communicating with Other Devices (continued)

- **Parallel Communication:** Several communication paths transfer bits simultaneously.
- **Serial Communication:** Bits are transferred one after the other over a single communication path.

Data Communication Rates

- Measurement units
 - Bps: Bits per second
 - Kbps: Kilo-bps (1,000 bps)
 - Mbps: Mega-bps (1,000,000 bps)
 - Gbps: Giga-bps (1,000,000,000 bps)
- Bandwidth: Maximum available rate

Other Architectures

- Technologies to increase throughput:
 - Pipelining: Overlap steps of the machine cycle
 - Parallel Processing: Use multiple processors simultaneously
 - SISD: No parallel processing
 - MIMD: Different programs, different data
 - SIMD: Same program, different data

1. Assume that the machine described in Appendix C uses memory-mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.
 - a. If register 7 contains the ASCII code for the letter A, what machine language instruction should be used to cause that letter to be printed at the printer?
 - b. If the machine executes a million instructions per second, how many times can this character be sent to the printer in one second?
 - c. If the printer is capable of printing five traditional pages of text per minute, will it be able to keep up with the characters being sent to it in (b)?
2. Suppose that the hard disk on your personal computer rotates at 3000 revolutions a minute, that each track contains 16 sectors, and that each sector contains 1024 bytes. Approximately what communication rate is required between the disk drive and the disk controller if the controller is going to receive bits from the disk drive as they are read from the spinning disk?

1. Assume that the machine described in Appendix C uses memory-mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.
 - a. If register 7 contains the ASCII code for the letter A, what machine language instruction should be used to cause that letter to be printed at the printer?
 - b. If the machine executes a million instructions per second, how many times can this character be sent to the printer in one second?
 - c. If the printer is capable of printing five traditional pages of text per minute, will it be able to keep up with the characters being sent to it in (b)?
2. Suppose that the hard disk on your personal computer rotates at 3000 revolutions a minute, that each track contains 16 sectors, and that each sector contains 1024 bytes. Approximately what communication rate is required between the disk drive and the disk controller if the controller is going to receive bits from the disk drive as they are read from the spinning disk?

1.
 - a. 37B5
 - b. One million times
 - c. No. A typical page of text contains less than 4000 characters. Thus the ability to print five pages in a minute indicates a printing rate of no more than 20,000 characters per minute, which is much less than one million characters per second. (The point is that a computer can send characters to a printer much faster than the printer can print them; thus the printer needs a way of telling the computer to wait.)

1. Assume that the machine described in Appendix C uses memory-mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.
 - a. If register 7 contains the ASCII code for the letter A, what machine language instruction should be used to cause that letter to be printed at the printer?
 - b. If the machine executes a million instructions per second, how many times can this character be sent to the printer in one second?
 - c. If the printer is capable of printing five traditional pages of text per minute, will it be able to keep up with the characters being sent to it in (b)?
2. Suppose that the hard disk on your personal computer rotates at 3000 revolutions a minute, that each track contains 16 sectors, and that each sector contains 1024 bytes. Approximately what communication rate is required between the disk drive and the disk controller if the controller is going to receive bits from the disk drive as they are read from the spinning disk?

2. The disk will make 50 revolutions in one second, meaning that 800 sectors will pass under the read/write head in a second. Because each sector contains 1024 bytes, bits will pass under the read/write head at approximately 6.5 Mbps. Thus communication between the controller and the disk drive will have to be at least this fast if the controller is going to keep up with the data being read from the disk.